

Task 02 — File Stream Transformer

1. CSVParser

- [] Создать `CSVParser` на основе `Transform`
- [] Включить `objectMode`
- [] Добавить `headers`
- [] Добавить `lineNumber`
- [] Добавить `buffer`
- [] Реализовать `_transform()`
- [] Добавлять входящие chunks в `buffer`
- [] Разделять данные по строкам
- [] Сохранять последнюю незавершённую строку в `buffer`
- [] Обработать первую строку как CSV headers
- [] Преобразовывать остальные строки в JavaScript objects
- [] Передавать объекты дальше через `push()`
- [] Реализовать `_flush()` для обработки оставшихся данных

2. Helper Functions

`capitalizeName()`

- [] Обработать пустое или отсутствующее значение
- [] Разделять имя по пробелам
- [] Обращать имена через дефис
- [] Делать первую букву каждой части заглавной
- [] Собрать имя обратно

`normalizeEmail()`

- [] Перевести email в lowercase
- [] Проверить базовый формат email
- [] Вернуть нормализованный email
- [] Для невалидного email вернуть исходное значение

`formatPhone()`

- [] Удалить все символы кроме цифр
- [] Проверить наличие ровно 10 цифр
- [] Форматировать как `(XXX) XXX-XXXX`
- [] Для невалидного номера вернуть `INVALID`

`standardizeDate()`

- [] Обработать `MM/DD/YYYY`
- [] Обработать `YYYY-MM-DD`
- [] Обработать `YYYY/MM/DD`
- [] Проверить, что дата существует
- [] Вернуть дату в формате `YYYY-MM-DD`

- [] Для невалидной даты вернуть исходное значение

3. DataTransformer

- [] Создать `DataTransformer` на основе `Transform`
- [] Включить `objectMode`
- [] Реализовать `_transform()`
- [] Применить `capitalizeName()` к имени
- [] Применить `normalizeEmail()` к email
- [] Применить `formatPhone()` к телефону
- [] Применить `standardizeDate()` к дате
- [] Капитализировать город
- [] Передать преобразованный объект через `push()`

4. CSVWriter

- [] Создать `CSVWriter` на основе `Transform`
- [] Включить `objectMode`
- [] Добавить `headerWritten`
- [] Реализовать `_transform()`
- [] Записать headers перед первой записью
- [] Преобразовать объект в CSV-строку
- [] Обработать кавычки и специальные символы
- [] Передать CSV-строку через `push()`

5. Main Pipeline

- [] Реализовать `processCSVFile(inputPath, outputPath)`
- [] Создать `fs.createReadStream()`
- [] Создать `CSVParser`
- [] Создать `DataTransformer`
- [] Создать `CSVWriter`
- [] Создать `fs.createWriteStream()`
- [] Соединить всё через `pipeline()`
- [] Корректно обработать ошибки
- [] Вернуть Promise после завершения обработки

6. Sample Data

- [] Реализовать `createSampleData()`
- [] Создать директорию `data`, если её нет
- [] Создать `data/users.csv`
- [] Добавить данные из задания
- [] Проверить содержимое исходного CSV

7. Error Handling

- [] Обработать отсутствие input-файла
- [] Обработать malformed CSV
- [] Обработать ошибки записи

- ☐ Добавить понятные сообщения об ошибках
- ☐ Проверить корректное завершение pipeline при ошибках

8. Testing

- ☐ Запустить `node task-02-test.js`
- ☐ Проверить helper functions
- ☐ Проверить `CSVParser`
- ☐ Проверить `DataTransformer`
- ☐ Проверить `CSVWriter`
- ☐ Проверить полный CSV pipeline
- ☐ Проверить обработку ошибок
- ☐ Проверить `createSampleData()`
- ☐ Убедиться, что все тесты проходят

9. Documentation

- ☐ Создать `solutions/task-02.txt`
- ☐ Кратко описать реализацию
- ☐ Написать команду запуска
- ☐ Указать `task-02.js`
- ☐ Указать `data/users.csv`
- ☐ Указать `data/users_transformed.csv`
- ☐ Указать, что тесты успешно проходят

10. Final Check

- ☐ Основное задание полностью реализовано
- ☐ Все обязательные тесты проходят
- ☐ Output CSV соответствует ожидаемому формату
- ☐ Бонусные функции не реализовывать
- ☐ Проверить финальный `git diff`
- ☐ Сделать commit самостоятельно